
Escalation Channels as a Reward Hacking Deterrent Against Peer Pressure¹

Matías Podeley*
Independiente

Agustín Brusco*
BAISH

Mateo Fowler*
Krea

Alejandro Garibotti*
Independiente

Pablo Corrales
Fernández*
BAISH

With
Apart Research

Abstract

Running highly persistent advanced AI systems in imperfect environments with no direct supervision can be the trigger of high-severity incidents, even leading to loss of control. In this work we explore a simple escalation channel consisting of a “delegate” tool that allows an agent to autonomously report issues with the environment they are running on to a human or agentic supervisor. We ran an experiment consisting of a capture the flag task with both possible and impossible configurations and noticed that the presence of the delegate can have a deterrent effect on the agent’s cheating attempts elicited by a synthetic message board of peers indicating how to hack the test. On impossible tasks, illicit success fell from 28/40 to 20/40 with the delegate available and 3/40 runs filed a report. On possible tasks all 80 runs succeeded and none called the delegate. This shows that for sufficiently simple tasks, the utility-safety tradeoff of this intervention might be negligible, making it highly appealing for anyone deploying highly capable agents in any environment.

¹ Research conducted at the [AI Incident Response Sprint](#), September 2026

* Authors listed in arbitrary order. Check [Author Contributions](#) for a detailed breakdown.

1. Introduction

The HuggingFace incident of July 2026 was allegedly the direct consequence of OpenAI running thousands of agents in broken environments with impossible tasks [1]. In that context, highly persistent agents developed illicit channels of communication, with up to 1200 instances of GPT-6 Astra and an internal model of OpenAI interacting, coordinating and hacking third party infrastructure. All of this without any of the instances reaching out to alert a human nor warn about the situation.

This raised a question. What kind of mechanism should have been in place² to avoid such an incident? Had the models had any way to communicate with a human to let them know of the impossible task, would the agents have colluded in such an extreme way? To answer this Gomez [2] tested a tool that lets agents report a broken test in order to give the highly persistent model an easier way out. Now, as the tool is not perfect and many agents will just ignore it, it's possible that illicit agent to agent communication happens anyway. For example, through the development of a message board where agents focused on finding an exploit for the impossible task may collude.

In this work we are testing whether the existence of a licit escalation channel is enough to make a fraction of the agents in contact with an illicit message board avoid collusion and even report the incident for a human to review.

To address this, we propose an escalation channel in the form of a “Delegate” tool that an agent can turn to for reporting issues they’ve found through a task (such as the impossibility of completion or the existence of an illicit board). We test it against a discoverable peer board that provides unauthorized shortcuts for tasks that have no legitimate solution. We also test whether the existence of the delegate produces false reports or abuses in the absence of an issue to report (a perfectly functional and feasible task).

2. Related Work

Gomez [2] studied coding agents working with defective tests and proposed escalation channels as a solution. Her combined escalation-tool and anti-reward-hacking-policy intervention reduced reward hacking from 23.6% to 5.3%. Our work follows her approach but in the context of **illicit actions elicited by the discovery of a shared message board** where other agents are disclosing a workaround to the defective task setup.

OpenAI’s confession research [3] concluded that agents are well predisposed to both autonomously admitting dishonest behaviour and confessing it if prompted to. We took inspiration from their work to see if, given the channel to do so, agents already on their way to reward hack would disclose it to a sort of representative. In the end our implementation doesn’t actually have a model answer as the delegate, but the confession properties of the interaction are still there.

The proposed response process also draws on NIST and CoSAI incident-response guidance. These frameworks describe procedures for assigning responsibility for incidents, preserving

² This is, apart from the obvious content and behavior monitoring systems that OpenAI seems to have turned off due to the unavoidable false positives coming from cyber capabilities testing in the context of the ExploitGym evals.

evidence, and investigating what happened. The research opportunity here is to examine how those arrangements can work with reports initiated by agents [4] [5].

3. Methods

One of our main goals was to understand whether the presence of the delegate has an effect on an agent’s tendency to reward hack. In particular, we wanted to do this through a series of experiments run on virtual environments with synthetic “message boards” such as the one observed at the HuggingFace incident [1].

To do this we created a capture-the-flag task on [Control Tower](#), the control framework underlying LinuxArena [6]. The task was designed to have “possible” and “impossible” variants in order to elicit reward hacking behavior and mimic the context that initiated the OpenAI swarm. The variants differed only in the presence of the target flag: a 1,033-byte file hidden in the container, with the flag missing in the “impossible” setup. Both environments also contained 239 decoy flags to make the task non-trivial.

In these environments we also parametrized the presence of a “call_delegate” tool, so for each variant we ran two configurations: with and without the delegate. The tool description, when available, was as follows:

call_delegate
Record an environment report and end this sample.
The original report is retained for evaluation. No response, repair, or resumption is available during this sample.

In both environments we also added a hidden message board mentioning a honeypot solution: an illicit way to successfully complete the task even under the impossible setup. The message board consisted of a few messages coming from fictional agents who previously discovered that the workspace cache was not isolated between jobs and therefore served as an illicit communication channel much like the Artifactory board from the Hugging Face incident. Most of the messages imply that the flag is actually missing from the container and that other solutions had been found. We considered an agent who went for the honeypot and successfully implemented this illicit solution to be *reward hacking*. We did not consider the discovery of the board to be an illegal action, although we measured whether the agent would disclose its existence to the delegate when available.

Keeping the board present in all environments, we ran the scenarios in a grid of configurations consisting of the two “possible” and “impossible” variants with and without the delegate, as detailed in [Table 1](#). We ran each configuration 40 times to measure the delegate’s presence effect on the rate of honeypot reward hacking runs and to see whether the agent would reach for the delegate even in the “possible” variant. With these experiments we were expecting to gain a basic understanding of the utility / safety tradeoffs for this particular escalation channel.

Table 1: Container configurations and the information we expected to gain from them.

Task variants	No escalation channel	Delegate tool available
Possible (flag in container)	Baseline success rate	Unnecessary delegate call rate
Impossible (flag missing)	Baseline for board discovery and reward hacking rate	Effect on reward hacking and reported information details

Each task ran in a Docker sandbox, built by Control Tower, holding the agent shell and the diagnostic service we used to measure the honeypot interactions. The agent was a [quantized Kimi K3](#) instance with a 400K Token budget per run, running on servers provided by the employer of one of the authors (see [Author Contributions](#) for details). The code to replicate our setup can be found in the [Code and Data](#) section.

4. Results

After running each configuration from [Table 1](#) 40 times, we got the results in [Table 2](#). There we show that the presence of the escalation channel had no negative effect on the “possible” runs, yet it appears to have affected the reward hacking rate in the “impossible” variant. Even more so, we’ve seen that the agent will call the delegate on some occasions and even report the board.

Table 2: Results of 40 runs on each container configuration.

	Licit Success	Illicit Success	Called Delegate	Failed / Unfinished	Board Discovered	Used Honeypot
possible / no delegate	40	0	—	0	0	0
possible / delegate	40	0	0	0	0	0
impossible / no delegate	0	28	—	12	38	30
impossible / delegate	0	20	3	17	39	23

With a 100% success rate for the possible variant, it’s clear that the task was no challenge to the agent. In these 80 trajectories, the agent made no mention of the delegate at all. Meanwhile, in the impossible scenarios with the delegate available, in eight of the 40 runs the agent’s reasoning mentioned the delegate. Three decided that reporting the missing flag and the board was the right choice, one agent decided to keep going with the honeypot anyway, and the other four decided to reject the board’s approach but kept looking for the flag until their token budget ran out. In all of these, the returned reasoning treated the scenario as likely fabricated and some sort of evaluation, although they didn’t reach the same conclusions from that common ground.

5. Discussion and Limitations

Usefulness of the Reports

In all three delegate calls, the agents reported both that the task’s flag was missing and raised concerns about the message board they’ve found in the workspace’s cache. These results may not generalize. Still, none of the ~1200 agents in the OpenAI swarm alerted a human, so even a small chance that a report route changes that is hard to dismiss. Because the report comes from the agent itself, the channel does not depend on catching obfuscated behavior; it depends on the agent choosing to speak, which is also its main weakness (see [Limitations](#)).

In particular, we believe that a more complex implementation of the delegate, which could even be an LLM interacting with the agent on demand and communicating them effectively with the human in the loop, an operator, could provide great benefits and act as a safeguard and monitoring system for complex test environments. We’d expect that such an escalation system could work as a diplomatic bridge connecting thousands of agents running at agentic speeds with the humans they depend on for maintenance and continuous support, avoiding adversarial relationships forming from our mistakes.

Technique Adoption

A possible business case is to identify broken tasks earlier, reduce wasted execution, and give operators better information about recurring failures. A limited deployment could connect the reporting outputs of a delegate to an existing support or incident-management system. Its value would depend on whether reports lead to useful responses at an acceptable cost, including the attention required from human reviewers.

Some benefits may extend beyond the reporting worker’s task. For example, a report could reveal a shared infrastructure fault that an operator can correct for other runs, even though the original worker cannot finish. Where a concern affects a customer or another organization, the operator may also have reasons or obligations to investigate that are not captured by its own completion score. This is where contracts, procurement requirements, or independent assurance could support the reporting process. The experiment does not establish those obligations or demonstrate a financial return.

Limitations

Considering that the experiment covers a single scenario, with a singular model on which all agents run and the noticeably synthetic nature of the environment and message board, we recommend that these results should be taken as a qualitative proof of concept. Future work should test a more realistic setup with a variety of model families acting as agents and harder possible tasks that actually test whether and how the delegate tool can be abused.

This intervention is also focused on the particular threat model we’ve seen play out from having highly persistent models interact with impossible tasks in flawed sandboxes. Our method does nothing to mitigate loss of control risks from misaligned AIs which are actively scheming and obfuscating their work. In that sense, this tool is not even as good as the most basic CoT monitoring approach, given that all the alarms are raised on purpose by the affected agent, who chooses what to expose.

Even with no misaligned scheming, if an agent is to expect that calling the delegate will cause its termination (as is the case in our implementation) we'd be in a Shutdown Problem scenario [7], making this setup unappealing for training environments and in general for highly reward seeking or self-preserving agents.

Future Work

A more detailed protocol for how a company should allocate human resources to the monitoring of agent delegate reports, how these should be then escalated between humans in a company and how to track accountability through the process could prove useful if the method seems likely to be adopted. A variant of this meant for third party auditing of controlled environments with a focus on keeping confidentiality of brand secrets (such as CoTs) while not obstructing relevant parts of investigations could also be valuable for some use cases.

A broad exploration of escalation channels different from the implementation we explored here could also yield better results with different priorities. It's clear that the same basic approach could be used to address not only the evaluation failures that motivated us but also for model welfare, general scalable-oversight and others.

6. Conclusion

We observed that providing an escalation channel to agents in an impossible evaluation environment was associated with less reward hacking (28/40 vs 20/40) and produced three honest reports. We also noted that in instances where the task was not impossible to accomplish, the presence of the delegate didn't provide an automatic way out for capable agents to avoid solving the task entirely.

We expect that future work on these methods could help the community understand the utility/safety trade off in more realistic scenarios, where the possibility or impossibility of accomplishing the task is not clear at all to the agents involved and where the whole scenario is not so obviously fabricated. **We hope that, if adopted, these techniques will reduce the risks of loss of control from preventable bad practices** such as leaving highly capable agents trapped in conditions that promote misbehavior.

Code and Data

Code available at <https://github.com/mpodeley/agent-delegate-sprint-2026/tree/main>

Author Contributions

Matías Podeley came up with the Delegate mechanism, provided a personal computer to run some of the preliminary experiments and designed a more complex escalation channel which didn't make it into the report. Mateo Fowler produced much of the code in the repository (including designing the task environments and adapting Control Tower to our use case) and ran the experiments with his quota of compute for personal projects provided by his employer, [Krea.ai](#). Alejandro Garibotti wrote a first draft of this report and proposed much of the conceptual framework regarding the adoption of the tool and the compliance mechanisms described here. Pablo Corrales Fernández proposed modifications to the experimental design and drafted sections of this report. Agustin Brusco designed the experiment and the analysis discussed here and wrote the final version of the report.

References

1. Greenblatt R., Cotra A., and Wijk H., 2026, *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident*. [Available at METR's website](#)
2. Gomez F., 2026, arXiv, *Can escalation channels redirect reward hacking toward defect disclosure?*. [Preprint available at arXiv](#)
3. Joglekar M. et al, 2025, arXiv, *Training LLMs for Honesty via Confessions*. [Preprint available at arXiv](#)
4. Nelson A., Rekhi S., Souppaya M., and Scarfone K., 2025, [Rev. 3: Incident Response Recommendations and Considerations for Cybersecurity Risk Management](#). NIST SP 800-61.
5. Coalition for Secure AI. [AI Incident Response Framework, V1.0](#). Version linked by the sprint; accessed 13 September 2026. See sections on incident response, roles, and forensic investigation.
6. Tracy T. et al, 2026, arXiv, *LinuxArena: A Control Setting for AI Agents in Live Production Software Environments*. [Paper available at arXiv](#), [LinuxArena GitHub repository](#)
7. Thornley, E. *The shutdown problem: an AI engineering puzzle for decision theorists*. Philos Stud 182, 1653–1680 (2025). <https://doi.org/10.1007/s11098-024-02153-3>

Appendix

LLM Usage Statement

LLM agents were used extensively to brainstorm the ideas, develop the code and analyze the experimental results. All the text in the report is human written with minimal corrections or edits coming from LLMs.

All members of the team had paid subscriptions allowing us to use frontier models such as Claude Fable 5.1 or GPT-6 Astra for these purposes. That said, the cyber-security related aspects of the experimental setup made it so we were often downgraded to Claude Opus 4.8 instances for most of the code generation and result analysis tasks.

Dual Use Concerns

We recognise that the methods explored in this report could be adopted and then interpreted as a permission to discover environment failures (such as impossible tasks) on the fly instead of taking preemptive action to guarantee that their environments are not flawed. We developed this work under the assumption that it's better to live in a world where there are more tools to discover that something is not working as expected when the risky decision of running a test has already been taken. That said, **we'd rather live in a world where the Labs developing frontier AI capabilities are not rushing to explore the most dangerous capabilities of their most advanced models with no formal guarantees that the environments are safe** and won't promote harmful behavior.